

Claude Code 사용하기

Claude가 프로젝트를 기억하는 방법

CLAUDE.md 파일로 Claude에 지속적인 지침을 제공하고, 자동 메모리를 통해 Claude가 자동으로 학습을 축적하도록 합니다.

각 Claude Code 세션은 새로운 컨텍스트 윈도우로 시작됩니다. 두 가지 메커니즘이 세션 간에 지식을 전달합니다:

- **CLAUDE.md 파일**: Claude에 지속적인 컨텍스트를 제공하기 위해 작성하는 지침
- **자동 메모리**: 수정 및 선호도에 따라 Claude가 자신을 위해 작성하는 노트

이 페이지에서 다루는 내용

- CLAUDE.md 파일 작성 및 구성
- `.claude/rules/` 를 사용하여 특정 파일 유형에 규칙 범위 지정
- 자동 메모리 구성하여 Claude가 자동으로 노트를 작성하도록 함
- 지침이 따라지지 않을 때 문제 해결

CLAUDE.md vs 자동 메모리

Claude Code에는 두 가지 상호 보완적인 메모리 시스템이 있습니다. **둘 다 모든 대화의 시작 시 로드**됩니다. Claude는 이들을 강제된 구성이 아닌 컨텍스트로 취급합니다. 작업을 차단하려면 어떤 Claude의 결정과 관계없이 PreToolUse 혹은 사용합니다. 지침이 더 구체적이고 간결할수록 Claude가 더 일관되게 따릅니다.

	CLAUDE.md 파일	자동 메모리
작성자	사용자	Claude
포함 내용	지침 및 규칙	학습 및 패턴
범위	프로젝트, 사용자 또는 조직	저장소당 , 작업 트리 전체에서 공유
로드 대상	모든 세션	모든 세션(처음 200줄 또는 25KB)
사용 목적	코딩 표준, 워크플로우, 프로젝트 아키텍처	빌드 명령, 디버깅 인사이트, Claude가 발견한 선호도

Claude의 동작을 안내하려면 CLAUDE.md 파일을 사용합니다. 자동 메모리를 통해 Claude는 수동 작업 없이 수정 사항에서 학습할 수 있습니다.

Subagent도 자신의 자동 메모리를 유지할 수 있습니다. 자세한 내용은 subagent 구성(`/docs/sub-agents#enable-persistent-memory`)을 참조하세요.

CLAUDE.md 파일

CLAUDE.md 파일은 프로젝트, 개인 워크플로우 또는 전체 조직에 대해 Claude에 지속적인 지침을 제공하는 마크다운 파일입니다. 이러한 파일을 일반 텍스트로 작성하면 Claude가 모든 세션의 시작 시 읽습니다.

CLAUDE.md에 추가할 시기

CLAUDE.md를 다시 설명해야 할 내용을 적어두는 장소로 취급합니다. 다음과 같은 경우에 추가합니다:

- Claude가 **같은 실수**를 두 번째로 합니다
- 코드 리뷰에서 Claude가 이 코드베이스에 대해 알아야 할 것을 발견합니다
- 지난 세션에 입력한 것과 같은 수정 또는 설명을 채팅에 입력합니다
- 새로운 팀원이 생산성을 높이기 위해 **같은 컨텍스트**가 **필요**합니다

모든 세션에서 Claude가 보유해야 할 사실로 유지합니다: 빌드 명령, 규칙, 프로젝트 레이아웃, "항상 X를 수행합니다" 규칙. **항목이 다 단계 절차**이거나 **코드베이스의 한 부분에만 중요한 경우** 대신 **skill** 또는 **경로 범위 규칙**으로 이동합니다.

CLAUDE.md 파일을 어디에 배치할지 선택

CLAUDE.md 파일은 여러 위치에 있을 수 있으며, 각각 다른 범위를 가집니다. 아래 표는 로드 순서대로 나열되어 있으며, 가장 광범위한 범위에서 가장 구체적인 범위까지이므로 프로젝트 지침이 사용자 지침 이후에 컨텍스트에 나타납니다.

사용자 지침 하나만 global 한 줄 알았는데, 관리 정책이 저장되는 위치와 목적이 따로 있다는 것을 처음 알게 됨!

범위	위치	목적	사용 사례	공유 대상
관리 정책	macOS: <code>/Library/Application Support/ClaudeCode/CLAUDE.md</code> Linux/WSL: <code>/etc/claude-code/CLAUDE.md</code> Windows: <code>C:\Program Files\ClaudeCode\CLAUDE.md</code>	IT/DevOps에서 관리하는 조직 전체 지침	회사 코딩 표준, 보안 정책, 규정 준수 요구 사항	조직의 모든 사용자
사용자 지침	<code>~/.claude/CLAUDE.md</code>	모든 프로젝트에 대한 개인 선호도	코드 스타일 선호도, 개인 도구 단축키	본인만(모든 프로젝트)
프로젝트 지침	<code>./CLAUDE.md</code> 또는 <code>./.claude/CLAUDE.md</code>	프로젝트에 대한 팀 공유 지침	프로젝트 아키텍처, 코딩 표준, 일반적인 워크플로우	소스 제어를 통한 팀 멤버
로컬 지침	<code>./CLAUDE.local.md</code>	개인 프로젝트별 선호도; <code>.gitignore</code> 에 추가	샌드박스 URL, 선호하는 테스트 데이터	본인만(현재 프로젝트)

작업 디렉토리 위의 디렉토리 계층 구조에 있는 CLAUDE.md 및 CLAUDE.local.md 파일은 시작 시 전체 로드됩니다. 하위 디렉토리의 파일은 Claude가 해당 디렉토리의 파일을 읽을 때 필요에 따라 로드됩니다.

대규모 프로젝트의 경우 프로젝트 규칙을 사용하여 지침을 주제별 파일로 나눌 수 있습니다. 규칙을 통해 특정 파일 유형 또는 하위 디렉토리에 지침의 범위를 지정할 수 있습니다.

프로젝트 CLAUDE.md 설정

프로젝트 CLAUDE.md는 `./CLAUDE.md` 또는 `./.claude/CLAUDE.md`에 저장할 수 있습니다. 이 파일을 만들고 프로젝트에서 작업하는 모든 사람에게 적용되는 지침을 추가합니다: **빌드 및 테스트 명령, 코딩 표준, 아키텍처 결정, 명령 규칙 및 일반적인 워크플로우**. 이러한 지침은 **버전 제어를 통해 팀과 공유**되므로 개인 선호도보다는 **프로젝트 수준의 표준**에 중점을 두세요.



팁

`/init` 을 실행하여 시작 CLAUDE.md를 자동으로 생성합니다. Claude는 코드베이스를 분석하고 발견한 빌드 명령, 테스트 지침 및 프로젝트 규칙이 포함된 파일을 만듭니다. CLAUDE.md가 이미 존재하면 `/init` 은 덮어쓰지 않고 개선 사항을 제안합니다.

`CLAUDE_CODE_NEW_INIT=1` 을 설정하여 대화형 단계에 흐름을 활성화합니다. `/init` 은 설정할 아티팩트를 묻습니다: CLAUDE.md 파일, skills 및 hooks. 그런 다음 subagent로 코드베이스를 탐색하고 후속 질문을 통해 간격을 채우며 파일을 작성하기 전에 검토 가능한 제안을 제시합니다.

`CLAUDE_CODE_NEW_INIT=1`
이 설정이 무엇이길래 꼭 따로 설정을 해줘야 하는가?

효과적인 지침 작성

CLAUDE.md 파일은 모든 세션의 시작 시 컨텍스트 윈도우에 로드되어 대화와 함께 토큰을 소비합니다. 강제된 구성이 아닌 컨텍스트이기 때문에 지침을 작성하는 방식이 Claude가 얼마나 안정적으로 따르는지에 영향을 미칩니다. 구체적이고 간결하며 잘 구조화된 지침이 가장 잘 작동합니다.

크기: CLAUDE.md 파일당 200줄 이하를 목표로 합니다. 더 긴 파일은 더 많은 컨텍스트를 소비하고 준수를 줄입니다. 지침이 커지면 경로 범위 규칙을 사용하여 Claude가 일치하는 파일로 작업할 때만 지침이 로드되도록 하여 노이즈를 줄이고 컨텍스트 공간을 절약할 수 있습니다. 가져오기를 사용하여 조직을 위해 콘텐츠를 분할할 수도 있지만, 가져온 파일은 여전히 로드되고 시작 시 컨텍스트 윈도우에 들어갑니다.

구조: 마크다운 헤더와 글머리 기호를 사용하여 관련 지침을 그룹화합니다. Claude는 독자와 같은 방식으로 구조를 스캔합니다: 구성된 섹션이 조밀한 단락보다 따르기 쉽습니다.

구체성: 검증할 수 있을 정도로 구체적인 지침을 작성합니다. 예를 들어:

- "코드를 제대로 포맷합니다"보다는 "2칸 들여쓰기 사용"
- "변경 사항을 테스트합니다"보다는 "커밋하기 전에 `npm test` 실행"
- "파일을 정리된 상태로 유지합니다"보다는 "API 핸들러는 `src/api/handlers/`에 있습니다"

일관성: 두 규칙이 서로 모순되면 Claude가 하나를 임의로 선택할 수 있습니다. CLAUDE.md 파일, 하위 디렉토리의 중첩된 CLAUDE.md 파일 및 `.claude/rules/` 을 정기적으로 검토하여 오래되었거나 충돌하는 지침을 제거합니다. 모노레포에서는 `claudeMdExcludes` 를 사용하여 작업과 관련이 없는 다른 팀의 CLAUDE.md 파일을 건너뛵니다.

추가 파일 가져오기

CLAUDE.md 파일은 `@path/to/import` 구문을 사용하여 추가 파일을 가져올 수 있습니다. 가져온 파일은 확장되어 참조하는 CLAUDE.md와 함께 시작 시 컨텍스트에 로드됩니다. CLAUDE.md에서 import한 추가 파일 까지 포함하여 200줄을 넘지 않는 것이 좋은지?

상대 경로와 절대 경로 모두 허용됩니다. 상대 경로는 작업 디렉토리가 아닌 가져오기를 포함하는 파일을 기준으로 해석됩니다. 가져온 파일은 최대 4개 홑의 깊이로 다른 파일을 재귀적으로 가져올 수 있습니다.

가져오기 구문은 마크다운 코드 스펠과 펜스 코드 블록을 건너뛵니다. CLAUDE.md에서 경로를 가져오지 않고 언급하려면 백틱으로 감싸세요: ``@README`` 를 작성하면 텍스트가 리터럴로 유지되고, 백틱 외부의 `@README` 는 파일을 가져옵니다.

README, package.json 및 워크플로우 가이드를 가져오려면 CLAUDE.md의 어디든지 `@` 구문으로 참조합니다:

프로젝트 개요는 @README를 참조하고 이 프로젝트의 사용 가능한 npm 명령은 @package.json을 참조합니다.

추가 지침

- git 워크플로우 @docs/git-instructions.md

개인 프로젝트별 선호도의 경우 프로젝트 루트에서 `CLAUDE.local.md` 를 만듭니다. 이는 `CLAUDE.md` 와 함께 로드되고 같은 방식으로 취급됩니다. 버전 제어에 커밋되지 않도록 `.gitignore` 에 `CLAUDE.local.md` 를 추가합니다. `/init` 을 실행하고 개인 옵션을 선택하면 자동으로 수행됩니다.

동일한 저장소의 여러 git worktree에서 작업하는 경우 gitignored `CLAUDE.local.md` 는 생성한 worktree에만 존재합니다. worktree 간에 개인 지침을 공유하려면 대신 홈 디렉토리에서 파일을 가져옵니다:

```
# 개인 선호도
- @~/ .claude/my-project-instructions.md
```

⚠ 경고

Claude Code가 프로젝트에서 외부 가져오기를 처음 만날 때 파일을 나열하는 승인 대화를 표시합니다. 거부하면 가져오기가 비활성화된 상태로 유지되고 대화가 다시 나타나지 않습니다.

지침을 구성하는 더 구조화된 접근 방식은 `.claude/rules/` 섹션을 참조하세요.

AGENTS.md

?? 이게 무슨말임? AGENTS.md는 언제 쓰이는거고, CLAUDE.md랑은 무슨 관계임?

Claude Code는 `CLAUDE.md` 를 읽으며 `AGENTS.md` 를 읽지 않습니다. **저장소가 이미 다른 코딩 에이전트에 AGENTS.md 를 사용하는 경우 CLAUDE.md 를 만들어 이를 가져오면 두 도구가 중복 없이 동일한 지침을 읽을 수 있습니다.** Claude 특정 지침을 가져오기 아래에 추가할 수도 있습니다. Claude는 가져온 파일을 세션 시작 시 로드한 다음 나머지를 추가합니다:

```
@AGENTS.md

## Claude Code

`src/billing/` 아래의 변경 사항에 대해 Plan Mode를 사용합니다.
```

심볼릭 링크도 작동합니다. Claude 특정 콘텐츠를 추가할 필요가 없으면:

```
ln -s AGENTS.md CLAUDE.md
```

Windows에서 심볼릭 링크를 만들려면 관리자 권한 또는 개발자 모드가 필요하므로 대신 `@AGENTS.md` 가져오기를 사용합니다.

이미 `AGENTS.md` 가 있는 저장소에서 `/init` 을 실행하면 이를 읽고 관련 부분을 생성된 `CLAUDE.md` 에 통합합니다. 또한 `.cursorrules` , `.devin/rules/` 및 `.windsurfrules` 과 같은 다른 도구 구성을 읽습니다.

CLAUDE.md 파일이 로드되는 방식

Claude Code는 현재 작업 디렉토리에서 디렉토리 트리를 따라 올라가며 CLAUDE.md 파일을 읽고 각 디렉토리를 확인합니다. 즉, `foo/bar/` 에서 Claude Code를 실행하면 `foo/bar/CLAUDE.md` , `foo/CLAUDE.md` 및 그 옆의 모든 `CLAUDE.local.md` 파일에서 지침을 로드합니다.

발견된 모든 파일은 **서로를 재정의하지 않고 컨텍스트에 연결**됩니다. 디렉토리 트리 전체에서 콘텐츠는 파일 시스템 루트에서 작업 디렉토리까지 순서대로 정렬됩니다. `foo/bar/` 예제의 경우 `foo/CLAUDE.md` 가 `foo/bar/CLAUDE.md` 보다 먼저 컨텍스트에 나타나므로 **Claude를 시작한 위치에 더 가까운 지침이 마지막에 읽힙니다.** 각 디렉토리 내에서 `CLAUDE.local.md` 는 `CLAUDE.md` 후에 추가되므로 개인 노트가 해당 수준에서 Claude가 읽는 마지막 것입니다.

Claude는 또한 현재 작업 디렉토리 아래의 하위 디렉토리에서 `CLAUDE.md` 및 `CLAUDE.local.md` 파일을 발견합니다. 시작 시 로드하는 대신 Claude가 해당 하위 디렉토리의 파일을 읽을 때 포함됩니다.

대규모 모노레포에서 작업하고 다른 팀의 `CLAUDE.md` 파일이 선택되는 경우 `claudeMdExcludes` 를 사용하여 건너뛵니다.

`CLAUDE.md` 파일의 블록 수준 HTML 주석(`<!-- maintainer notes -->`)은 콘텐츠가 Claude의 컨텍스트에 주입되기 전에 제거됩니다. 컨텍스트 토큰을 소비하지 않고 인간 유지보수자를 위한 노트를 남기는 데 사용합니다. 코드 블록 내의 주석은 보존됩니다. Read 도구로 `CLAUDE.md` 파일을 직접 열 때 주석이 표시된 상태로 유지됩니다.

추가 디렉토리에서 로드

`--add-dir` 플래그는 Claude에 주 작업 디렉토리 외부의 추가 디렉토리에 대한 액세스를 제공합니다. 기본적으로 이러한 디렉토리의 `CLAUDE.md` 파일은 로드되지 않습니다.

추가 디렉토리에서 메모리 파일을 로드하려면 `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD` 환경 변수를 설정합니다:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1 claude --add-dir ../shared-config
```

이는 추가 디렉토리에서 `CLAUDE.md`, `.claude/CLAUDE.md`, `.claude/rules/*.md` 및 `CLAUDE.local.md` 를 로드합니다. `--setting-sources` 에서 `local` 을 제외하면 `CLAUDE.local.md` 는 건너뛵니다.

.claude/rules/로 규칙 구성

대규모 프로젝트의 경우 `.claude/rules/` 디렉토리를 사용하여 지침을 여러 파일로 구성할 수 있습니다. 이렇게 하면 지침이 모듈식이 되고 팀이 유지 관리하기 쉬워집니다. 규칙을 특정 파일 경로로 범위 지정할 수도 있으므로 Claude가 일치하는 파일로 작업할 때만 컨텍스트에 로드되어 노이즈를 줄이고 컨텍스트 공간을 절약합니다.

참고

규칙은 모든 세션 또는 일치하는 파일이 열릴 때 컨텍스트에 로드됩니다. 항상 컨텍스트에 있을 필요가 없는 작업별 지침의 경우 대신 `skills`를 사용하세요. 이는 호출할 때 또는 Claude가 프롬프트와 관련이 있다고 판단할 때만 로드됩니다.

규칙 설정

프로젝트의 `.claude/rules/` 디렉토리에 마크다운 파일을 배치합니다. 각 파일은 `testing.md` 또는 `api-design.md` 와 같은 **설명적인 파일명으로 한 가지 주제를 다루어야 합니다**. 모든 `.md` 파일은 재귀적으로 발견되므로 `frontend/` 또는 `backend/` 와 같은 하위 디렉토리로 규칙을 구성할 수 있습니다:

```
your-project/
├── .claude/
│   ├── CLAUDE.md          # 주 프로젝트 지침
│   └── rules/
│       ├── code-style.md  # 코드 스타일 가이드라인
│       ├── testing.md     # 테스트 규칙
│       └── security.md    # 보안 요구사항
```

`paths` frontmatter가 없는 규칙은 `.claude/CLAUDE.md` 와 동일한 우선순위로 시작 시 로드됩니다.

경로별 규칙

규칙은 `paths` 필드가 있는 `YAML frontmatter`를 사용하여 특정 파일로 범위를 지정할 수 있습니다. 이러한 조건부 규칙은 Claude가 지정된 패턴과 일치하는 파일로 작업할 때만 적용됩니다.

```
---
paths:
  - "src/api/**/*.ts"
---
```

API 개발 규칙

- 모든 API 엔드포인트는 입력 검증을 포함해야 합니다
- 표준 오류 응답 형식을 사용합니다
- OpenAPI 문서 주석을 포함합니다

`paths` 필드가 없는 규칙은 **무조건 로드**되며 모든 파일에 적용됩니다. 경로 범위 규칙은 모든 도구 사용 시가 아니라 Claude가 패턴과 일치하는 파일을 읽을 때 트리거됩니다. v2.1.198 이상에서는 예를 들어 프로젝트 디렉토리에 대한 심볼릭 링크된 경로를 통해 Claude가 파일에 도달할 때도 일치가 작동합니다.

`paths` 필드에서 glob 패턴을 사용하여 확장명, 디렉토리 또는 조합으로 파일을 일치시킵니다:

패턴	일치
<code>**/*.ts</code>	모든 디렉토리의 모든 TypeScript 파일
<code>src/**/*.ts</code>	<code>src/</code> 디렉토리 아래의 모든 파일
<code>*.md</code>	프로젝트 루트의 마크다운 파일
<code>src/components/*.tsx</code>	특정 디렉토리의 React 컴포넌트

여러 패턴을 지정하고 중괄호 확장을 사용하여 한 패턴에서 여러 확장명을 일치시킬 수 있습니다:

```
---
paths:
  - "src/**/*.{ts,tsx}"
  - "lib/**/*.ts"
  - "tests/**/*.test.ts"
---
```

Glob 구문은 `[`를 `[abc]`와 같은 괄호 표현식의 시작으로 취급합니다. `photos [2024/**`와 같이 괄호 표현식으로 읽을 수 없는 `[`가 있는 패턴은 유효하지 않습니다: 아무것도 일치하지 않으며 규칙의 다른 패턴은 계속 작동합니다. 파일 이름에서 리터럴 `[`를 일치시키려면 `photos \[2024/**`로 이스케이프합니다. v2.1.207 이전에는 하나의 유효하지 않은 패턴으로 인해 규칙이 평가된 모든 파일에 대해 Read 도구가 실패했으며, 대신 아무것도 일치하지 않습니다.

심볼릭 링크로 프로젝트 간 규칙 공유

`.claude/rules/` 디렉토리는 심볼릭 링크를 지원하므로 공유 규칙 세트를 유지하고 여러 프로젝트에 링크할 수 있습니다. 심볼릭 링크는 해석되어 정상적으로 로드되며 순환 심볼릭 링크는 감지되고 우아하게 처리됩니다.

이 예제는 공유 디렉토리와 개별 파일을 모두 링크합니다:

```
ln -s ~/shared-claude-rules .claude/rules/shared
ln -s ~/company-standards/security.md .claude/rules/security.md
```

사용자 수준 규칙

`~/.claude/rules/`의 개인 규칙은 컴퓨터의 모든 프로젝트에 적용됩니다. 프로젝트별이 아닌 선호도에 사용됩니다:

```
~/.claude/rules/
├─ preferences.md    # 개인 코딩 선호도
└─ workflows.md      # 선호하는 워크플로우
```

사용자 수준 규칙은 프로젝트 규칙 전에 로드되어 프로젝트 규칙에 더 높은 우선순위를 제공합니다.

대규모 팀을 위한 CLAUDE.md 관리

조직에서 Claude Code를 팀 전체에 배포하는 경우 지침을 중앙 집중식으로 관리하고 로드되는 CLAUDE.md 파일을 제어할 수 있습니다.

조직 전체 CLAUDE.md 배포

조직은 컴퓨터의 모든 사용자에게 적용되는 중앙 집중식으로 관리되는 CLAUDE.md를 배포할 수 있습니다. 이 파일은 개별 설정으로 제외될 수 없습니다.

1 관리 정책 위치에서 파일 만들기

- macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`
- Linux 및 WSL: `/etc/claude-code/CLAUDE.md`
- Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`

2 구성 관리 시스템으로 배포

MDM, 그룹 정책, Ansible 또는 유사한 도구를 사용하여 개발자 컴퓨터 전체에 파일을 배포합니다. 다른 조직 전체 구성 옵션은 관리 설정을 참조하세요.

`claudeMd` 키를 사용하면 별도의 파일을 배포하는 대신 관리 CLAUDE.md 콘텐츠를 `managed-settings.json`에 직접 배치할 수 있습니다.

범위: 컴퓨터의 모든 Claude Code 세션, 모든 저장소에서. 저장소별 지침은 대신 프로젝트 CLAUDE.md를 커밋합니다.

우선순위: 관리 CLAUDE.md 파일과 동일합니다. 사용자 및 프로젝트 CLAUDE.md 전에 로드됩니다.

적용되는 위치: 관리 및 정책 설정만. 사용자, 프로젝트 또는 로컬 설정에서 `claudeMd`를 설정해도 효과가 없습니다.

아래 예제는 관리 설정 파일에 행동 지침을 직접 추가합니다:

```
{
  "claudeMd": "Always run `make lint` before committing.\nNever push directly to main."
}
```

관리 CLAUDE.md와 관리 설정은 다른 목적을 제공합니다. 기술적 강제를 위해 설정을 사용하고 CLAUDE.md를 행동 지침으로 사용합니다:

관심사	구성 대상
특정 도구, 명령 또는 파일 경로 차단	관리 설정: <code>permissions.deny</code>
샌드박스 격리 강제	관리 설정: <code>sandbox.enabled</code>
환경 변수 및 API 공급자 라우팅	관리 설정: <code>env</code>
인증 방법 및 조직 잠금	관리 설정: <code>forceLoginMethod</code> , <code>forceLoginOrgUUID</code>
코드 스타일 및 품질 가이드라인	관리 CLAUDE.md
데이터 처리 및 규정 준수 알림	관리 CLAUDE.md
Claude의 행동 지침	관리 CLAUDE.md

설정 규칙은 Claude가 무엇을 하기로 결정하든 클라이언트에 의해 강제됩니다. CLAUDE.md 지침은 Claude의 행동을 형성하지만 하드 강제 레이어가 아닙니다.

특정 CLAUDE.md 파일 제외

대규모 모노레포에서 상위 CLAUDE.md 파일에는 작업과 관련이 없는 지침이 포함될 수 있습니다. `claudeMdExcludes` 설정을 통해 경로 또는 glob 패턴으로 특정 파일을 건너뛸 수 있습니다.

이 예제는 상위 폴더의 최상위 CLAUDE.md 및 규칙 디렉토리를 제외합니다. 제외가 컴퓨터에 로컬로 유지되도록 `.claude/settings.local.json`에 추가합니다:

```
{
  "claudeMdExcludes": [
    "**/monorepo/CLAUDE.md",
    "/home/user/monorepo/other-team/.claude/rules/**"
  ]
}
```

패턴은 glob 구문을 사용하여 절대 파일 경로와 일치합니다. `claudeMdExcludes`를 설정 레이어: 사용자, 프로젝트, 로컬 또는 관리 정책에서 구성할 수 있습니다. 배열은 레이어 전체에서 병합됩니다.

관리 정책 CLAUDE.md 파일은 제외될 수 없습니다. 이렇게 하면 개별 설정에 관계없이 조직 전체 지침이 항상 적용됩니다.

자동 메모리

자동 메모리를 통해 Claude는 아무것도 작성하지 않고도 세션 간에 지식을 축적할 수 있습니다. Claude는 작업할 때 자신을 위해 노트를 저장합니다: 빌드 명령, 디버깅 인사이트, 아키텍처 노트, 코드 스타일 선호도 및 워크플로우 습관. Claude는 모든 세션마다 뭔가를 저장하지 않습니다. 정보가 향후 대화에서 유용할지 여부에 따라 기억할 가치가 있는지 결정합니다.

자동 메모리 활성화 또는 비활성화

자동 메모리는 기본적으로 켜져 있습니다. 토크하려면 세션에서 `/memory`를 열고 자동 메모리 토글을 사용하거나 프로젝트 설정에서 `autoMemoryEnabled`를 설정합니다:

```
{
  "autoMemoryEnabled": false
}
```

환경 변수를 통해 자동 메모리를 비활성화하려면 `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1` 을 설정합니다.

저장소 위치

각 프로젝트는 `~/.claude/projects/<project>/memory/` 에서 자신의 메모리 디렉토리를 가집니다. `<project>` 경로는 git 저장소에서 파생되므로 동일한 저장소 내의 모든 worktree 및 하위 디렉토리는 하나의 자동 메모리 디렉토리를 공유합니다. git 저장소 외부에서는 프로젝트 루트가 대신 사용됩니다.

자동 메모리를 다른 위치에 저장하려면 `settings.json` 에서 `autoMemoryDirectory` 를 설정합니다. 이는 모든 설정 범위에서 유효합니다: 사용자, 프로젝트, 로컬, 정책 또는 `--settings` .

```
{
  "autoMemoryDirectory": "~/my-custom-memory-dir"
}
```

값은 절대 경로이거나 `~/` 로 시작해야 합니다. 프로젝트의 `.claude/settings.json` 또는 `.claude/settings.local.json` 에서 설정할 때, 값은 해당 폴더에 대한 워크스페이스 신뢰 대화를 수락한 후에만 적용됩니다. 이는 hook을 관리하는 것과 동일한 게이트입니다.

디렉토리에는 `MEMORY.md` 진입점과 선택적 주제 파일이 포함됩니다:

```
~/.claude/projects/<project>/memory/
├── MEMORY.md          # 간결한 인덱스, 모든 세션에 로드됨
├── debugging.md       # 디버깅 패턴에 대한 자세한 노트
├── api-conventions.md # API 설계 결정
└── ...                # Claude가 만드는 다른 주제 파일
```

`MEMORY.md` 는 메모리 디렉토리의 인덱스 역할을 합니다. Claude는 세션 전체에서 이 디렉토리의 파일을 읽고 쓰며 `MEMORY.md` 를 사용하여 저장된 내용을 추적합니다.

자동 메모리는 컴퓨터 로컬입니다. 동일한 git 저장소 내의 모든 worktree 및 하위 디렉토리는 하나의 자동 메모리 디렉토리를 공유합니다. 파일은 컴퓨터 간 또는 클라우드 환경 간에 공유되지 않습니다.

작동 방식

`MEMORY.md` 의 처음 200줄 또는 처음 25KB(둘 중 먼저 오는 것)는 모든 대화의 시작 시 로드됩니다. 해당 임계값을 초과하는 콘텐츠는 세션 시작 시 로드되지 않습니다. Claude는 자세한 노트를 별도의 주제 파일로 이동하여 `MEMORY.md` 를 간결하게 유지합니다. 이 제한은 `MEMORY.md`에만 적용됩니다. `CLAUDE.md` 파일은 길이에 관계없이 전체 로드되지만, 더 짧은 파일이 더 나은 준수를 생성합니다.

`debugging.md` 또는 `patterns.md` 와 같은 주제 파일은 시작 시 로드되지 않습니다. Claude는 필요한 정보가 필요할 때 표준 파일 도구를 사용하여 필요에 따라 읽습니다. 주제 파일이라는 것을 처음 알게 됨.
↳ `debugging.md`, `patterns.md` 말고 어떤 것들이 또 있을까?

Claude는 세션 중에 메모리 파일을 읽고 씁니다. Claude Code 인터페이스에서 "Writing memory" 또는 "Recalled memory"를 보면 Claude가 `~/.claude/projects/<project>/memory/` 에서 활발히 업데이트하거나 읽고 있습니다.

메모리 감사 및 편집

이걸 커스터마이징(편집)하는게 성능에 과연 좋을런지? 궁금함.

자동 메모리 파일은 언제든지 편집하거나 삭제할 수 있는 일반 마크다운입니다. `/memory` 를 실행하여 세션 내에서 메모리 파일을 찾아보고 엽니다.

`/memory` 로 보기 및 편집

`/memory` 명령은 현재 세션에 로드된 모든 CLAUDE.md, CLAUDE.local.md 및 규칙 파일을 나열하고, 자동 메모리를 켜거나 끌 수 있으며, 자동 메모리 폴더를 열 수 있는 링크를 제공합니다. 파일을 선택하여 편집기에서 엽니다.

Claude에게 "항상 npm이 아닌 pnpm을 사용합니다" 또는 "API 테스트에 로컬 Redis 인스턴스가 필요하다는 것을 기억합니다"와 같이 뭔가를 기억하도록 요청하면 Claude는 자동 메모리에 저장합니다. 대신 CLAUDE.md에 지침을 추가하려면 Claude에게 직접 "이것을 CLAUDE.md에 추가합니다"라고 요청하거나 `/memory` 를 통해 파일을 직접 편집합니다.

메모리 문제 해결

이들은 CLAUDE.md 및 자동 메모리의 가장 일반적인 문제와 디버깅 단계입니다.

Claude가 CLAUDE.md를 따르지 않습니다

CLAUDE.md 콘텐츠는 시스템 프롬프트의 일부가 아니라 시스템 프롬프트 후 사용자 메시지로 전달됩니다. Claude는 이를 읽고 따르려고 하지만 특히 모호하거나 충돌하는 지침의 경우 엄격한 준수를 보장하지 않습니다.

디버깅하려면:

- `/memory` 를 실행하여 CLAUDE.md 및 CLAUDE.local.md 파일이 로드되는지 확인합니다. 파일이 나열되지 않으면 Claude가 볼 수 없습니다.
- 관련 CLAUDE.md가 세션에 대해 로드되는 위치에 있는지 확인합니다.
- 지침을 더 구체적으로 만듭니다. "2칸 들여쓰기 사용"이 "코드를 제대로 포맷합니다"보다 더 잘 작동합니다.
- CLAUDE.md 파일 전체에서 충돌하는 지침을 찾습니다. 두 파일이 동일한 동작에 대해 다른 지침을 제공하면 Claude가 하나를 임의로 선택할 수 있습니다.

hook으로 작성되어있으면 지침에는 따로 추가하지 않아도 되는 건지?

명령이 모든 커밋 전이나 각 파일 편집 후와 같이 특정 시점에 실행되어야 하는 경우, 대신 hook으로 작성합니다. Hook은 고정된 라이프사이클 이벤트에서 셸 명령으로 실행되며 Claude가 무엇을 하기로 결정하든 관계없이 적용됩니다.

시스템 프롬프트 수준의 지침의 경우 `--append-system-prompt` 를 사용합니다. 이는 모든 호출 시 전달되어야 하므로 대화형 사용보다는 스크립트 및 자동화에 더 적합합니다. **예를들면? 시스템 프롬프트 수준의 지침이 무엇인지 와닿지 않음**



InstructionsLoaded hook을 사용하여 로드된 지침 파일, 로드 시기 및 이유를 정확히 기록합니다. 이는 경로별 규칙 또는 하위 디렉토리의 지연 로드 파일을 디버깅하는 데 유용합니다.

자동 메모리가 저장한 내용을 모릅니다

`/memory` 를 실행하고 자동 메모리 폴더를 선택하여 Claude가 저장한 내용을 찾아봅니다. 모든 것이 읽고, 편집하거나 삭제할 수 있는 일반 마크다운입니다.

CLAUDE.md가 너무 큼니다

200줄을 초과하는 파일은 더 많은 컨텍스트를 소비하고 준수를 줄일 수 있습니다. 경로별 규칙을 사용하여 Claude가 일치하는 파일로 작업할 때만 지침을 로드하거나 모든 세션에서 필요하지 않은 콘텐츠를 정리합니다. @path 가져오기로 분할하면 조직화에 도움이 되지만 가져온 파일이 시작 시 로드되므로 컨텍스트를 줄이지는 않습니다.

/doctor 점검은 체크인된 CLAUDE.md에 대한 정리를 제안합니다: 디렉토리 레이아웃, 종속성 목록, 아키텍처 개요와 같이 Claude가 코드베이스에서 파생할 수 있는 콘텐츠를 제거하고 도구 기본값과 다른 함정, 근거 및 규칙을 유지합니다. 정리 확인에는 Claude Code v2.1.206 이상이 필요합니다.

/compact 후 지침이 손실된 것 같습니다

프로젝트 루트 CLAUDE.md는 압축을 완전히 생존합니다: /compact 후 Claude는 디스크에서 CLAUDE.md를 다시 읽고 세션에 새로 다시 주입합니다. 하위 디렉토리의 중첩된 CLAUDE.md 파일은 자동으로 다시 주입되지 않습니다. 해당 하위 디렉토리의 파일을 다시 읽을 때 다음에 다시 로드됩니다.

압축 후 지침이 사라진 경우 CLAUDE.md에 작성되지 않고 대화에서만 제공되었거나 아직 다시 로드되지 않은 중첩된 CLAUDE.md에 있습니다. 세션 간에 지속되도록 CLAUDE.md에 대화 전용 지침을 추가합니다.

관련 리소스

- **구성 디버깅:** CLAUDE.md 또는 설정이 적용되지 않는 이유 진단
- **Skills:** 필요에 따라 로드되는 반복 가능한 워크플로우 패키지
- **설정:** 설정 파일로 Claude Code 동작 구성
- **Subagent 메모리:** subagent가 자신의 자동 메모리를 유지하도록 허용

출처: Claude Code 공식 문서 — code.claude.com/docs/ko/memory (개인 학습용으로 정리한 PDF)